

Manual Técnico de la Arquitectura: Sistema de Verificación de Identidad y Detección de Fraude en Cédulas por WhatsApp

Esteban Mercado, Juan David Ocampo y Camilo

Resumen—Este documento describe formalmente la arquitectura de software, el diseño del grafo de estados conversacional, el modelo de datos persistido en Supabase, la integración con las API de AWS (Lambda y Rekognition), y los modelos de análisis de métricas para la detección de fraude mediante fotos de la cédula colombiana recibidas a través del canal de WhatsApp. El sistema combina técnicas de procesamiento de lenguaje natural generativo (Google Gemini 2.5 Flash) y machine learning de visión artificial (AWS Rekognition) para automatizar el veredicto de autenticidad en tiempo real.

1. INTRODUCCIÓN

La suplantación de identidad y la presentación de documentos falsificados (tales como fotocopias a color, capturas en pantallas o impresiones de alta fidelidad) constituyen uno de los mayores vectores de fraude en las industrias financiera y de tecnología. El presente software, denominado **Detection Attacks**, propone un enfoque robusto que combina la agilidad y accesibilidad de la aplicación de mensajería **WhatsApp** como interfaz conversacional, con el poder analítico de la computación en la nube.

1.1. Alcance del Sistema

El sistema guía al usuario de manera intuitiva para que proporcione secuencialmente las fotos de la parte frontal y trasera de su cédula colombiana. Una vez capturadas, el motor de orquestación coordina la descarga de las imágenes desde los servidores de Meta, las transfiere a un microservicio en AWS Lambda y realiza la validación técnica.

1.2. Objetivos del Software

- Accesibilidad:** Permitir a cualquier ciudadano validar su identidad a través de WhatsApp sin necesidad de instalar aplicaciones adicionales.
- Automatización:** Retornar una clasificación de veracidad (Real vs Fraude/Fotocopia) en menos de 5 segundos.
- Telemetría y Monitoreo:** Registrar cada etapa del flujo con tiempos de ejecución e indicadores clave para su posterior explotación en un panel web interactivo.

2. ARQUITECTURA DEL SISTEMA

El sistema **Detection Attacks** adopta una arquitectura de microservicios distribuida e híbrida, combinando la flexibilidad de la orquestación en la nube (Render) con el procesamiento especializado en infraestructura de Amazon Web Services (AWS) y servidores virtuales (EC2).

2.1. Capa Orquestadora: FastAPI y LangGraph

El backend principal está desarrollado en **FastAPI** y se despliega en Render. Se encarga de recibir los webhooks de Meta, procesar los eventos e interactuar de forma conversacional con el usuario por WhatsApp. La orquestación conversacional es controlada por **LangGraph**, definiendo un grafo de estados determinista guiado por la estructura de datos **AgentState**:

```
1 class AgentState(TypedDict):
2     phone: str
3     input_text: str | None
4     media_id: str | None
5     current_state: str # GREETING,
6     AWAITING_FRONT, AWAITING_BACK, DONE
7     front_result: dict | None
8     back_result: dict | None
9     response: str
10    latency_aws_ms: int | None
11    latency_gemini_ms: int | None
12    node_executed: str | None
13    status: str
14    is_photocopy: bool
15    lambda_score: float | None
16    lambda_confidence: float | None
17    lambda_response: dict | None
```

Listing 1. Definición de **AgentState** en el Orquestador

2.2. Lógica de Transición de Estados

El flujo conversacional sigue una máquina de estados determinista guiada por la entrada del usuario:

1. ****GreetingNode:**** Saluda al usuario y solicita la foto frontal.
2. ****ProcessFrontNode:**** Descarga la imagen frontal de WhatsApp, invoca la API Gateway de AWS y transiciona a 'AWAITING_BACK' si el documento es válido.
3. ****ProcessBackNode:**** Descarga la imagen trasera, invoca la API Gateway, realiza la consolidación del veredicto final y transiciona a 'DONE'.
4. ****ResetNode:**** Limpia los datos de sesión y reinicia el flujo cuando el usuario escribe *reiniciar*.
5. ****UnsupportedTextNode:**** Responde con asistencia conversacional cuando se reciben mensajes fuera de la secuencia del flujo.

2.3. Capa de Análisis: AWS Lambda y Servidor EC2

El análisis técnico de las imágenes para determinar la veracidad del documento y detectar fraudes físicos involucra tres componentes en AWS:

2.3.1. Lambda Proxy: receiveCodedImage

Expuesta mediante ****Amazon API Gateway****, esta función actúa como un canalizador y formateador:

- Recibe la imagen de la cédula en formato Base64 desde el orquestador de Render.
- Remueve la cabecera del formato Base64 si existe y convierte el contenido en un buffer binario.
- Realiza una petición HTTP 'POST' multipart/form-data a la URL del servidor EC2 ('/evaluate/').
- Retorna al orquestador el veredicto consolidado ('score' y 'detection').

2.3.2. Clasificador de Fraude en Instancia EC2

La inteligencia de clasificación reside en una ****Instancia EC2**** dedicada:

- Expone un servidor web (puerto '8001') con el endpoint '/evaluate/'.
- Ejecuta un modelo entrenado para la detección de fraude físico y anomalías en fotos.
- Retorna un 'score' probabilístico continuo de fraude ($S \in [0, 1]$) y un objeto 'detection' con metadatos espaciales (grados de rotación, dimensiones de recorte y coordenadas).

2.3.3. Lambdas Auxiliares de Soporte

Adicionalmente, el entorno de desarrollo y almacenamiento cuenta con dos Lambdas auxiliares:

- ****lambda-s3-save:**** Recibe la imagen en Base64, analiza los *magic bytes* del archivo para identificar su extensión (MIME Type: 'image/jpeg', 'image/png', etc.), genera un identificador UUID único, y sube el archivo a un bucket de ****Amazon S3**** ('S3_BUCKET_NAME'), retornando la URL pública estructurada.
- ****lambda-rekognition:**** Permite la detección general de etiquetas sobre la imagen invocando el comando 'DetectLabelsCommand' de ****AWS Rekognition**** directamente en memoria (sin requerir S3) para identificar la presencia de objetos clave con una confianza mínima del 70 %.

2.4. Capa Conversacional y Humanización: Google Gemini 2.5 Flash

El orquestador utiliza la API de ****Google Gemini 2.5 Flash**** para humanizar las respuestas. Gemini recibe el prompt de contexto técnico (el JSON retornado por AWS que contiene el 'score' de EC2) y genera un mensaje amigable, libre de tecnicismos complejos y formateado para WhatsApp en español. Para prevenir el truncado del texto, se configuró un límite de 'max_output_tokens = 1000' en la configuración del modelo.

3. MODELO DE DATOS Y PERSISTENCIA

El sistema utiliza una base de datos relacional **PostgreSQL** alojada en **Supabase** para mantener el estado de las conversaciones y registrar los eventos históricos para su análisis estadístico.

3.1. Esquema de Tablas

La base de datos consta de dos tablas principales: **whatsapp_sessions** y **validation_logs**.

3.1.1. Tabla: whatsapp_sessions

Esta tabla almacena el estado actual de la sesión del usuario. Se indexa por el número de teléfono del cliente para garantizar transiciones de estado rápidas.

3.1.2. Tabla: validation_logs

Esta tabla registra cada una de las interacciones y transacciones procesadas por el bot. Es la fuente de datos principal para el Dashboard de monitoreo.

Tabla 1
Atributos de la tabla whatsapp_sessions

Columna	Tipo	Descripción
phone	TEXT (PK)	Número de teléfono del usuario.
state	TEXT	Estado actual de la máquina de estados.
front_result	JSONB	Metadatos técnicos de la cara frontal.
back_result	JSONB	Metadatos técnicos de la cara trasera.
updated_at	TIMESTAMPTZ	Fecha y hora del último mensaje.

Tabla 2
Atributos de la tabla validation_logs

Columna	Tipo	Descripción
id	SERIAL (PK)	Identificador único correlativo.
phone	TEXT	Número de teléfono del usuario.
timestamp	TIMESTAMP-TZ	Fecha y hora del procesamiento.
node	TEXT	Nombre del nodo ejecutado.
whatsapp_message_id	TEXT	Identificador wamid de Meta.
lambda_score	NUMERIC	Score de fraude obtenido.
lambda_confidence	NUMERIC	Confianza de detección de cédula.
lambda_response	JSONB	Respuesta JSON completa de AWS.
latency_aws_ms	INTEGER	Latencia del API de AWS (ms).
latency_gemini_ms	INTEGER	Latencia del API de Gemini (ms).
status	TEXT	Estado del resultado de la petición.
is_photocopy	BOOLEAN	Indicador binario de fraude.

3.2. Restricciones de Pooling en Supabase

Dado que Supabase implementa un Connection Pooler (PgBouncer) configurado en **modo transacción** para manejar altas concurrencias (puerto 6543), se configuró explícitamente el parámetro `statement_cache_size=0` en la librería `asyncpg` en el backend.

Esta configuración es obligatoria para evitar errores de ejecución en la base de datos debido al reuso indebido de prepared statements a nivel de sesión en el pooler.

4. MODELO DE MÉTRICAS Y FÓRMULAS MATEMÁTICAS

Las métricas mostradas en el Dashboard de Detection Attacks se derivan de la consolidación de los registros históricos en la tabla 'validation_logs'. Esta sección detalla la construcción conceptual y matemática de dichos indicadores.

4.1. Criterio de Clasificación de Fraude

El sistema asigna un veredicto binario sobre la veracidad del documento basándose en la puntuación de confianza S retornada por el microservicio de AWS Rekognition. El modelo está calibrado para que $S \in [0, 1]$, donde un valor cercano a 0 representa un documento real y un valor cercano a 1 indica una sospecha o intento de fraude.

Definimos el veredicto de fraude como una función indicadora I_{fraude} con un umbral de decisión $\tau = 0,5$:

$$I_{\text{fraude}}(S) = \begin{cases} 1 & \text{si } S \geq 0,5 \quad (\text{FRAUDE} / \text{Sospecha}) \\ 0 & \text{si } S < 0,5 \quad (\text{REAL} / \text{Aprobado}) \end{cases} \quad (1)$$

Este umbral coincide con la línea de decisión visible en los histogramas de distribución del dashboard.

4.2. Indicadores Clave de Rendimiento (KPIs)

Sea N_{total} el número total de transacciones registradas en el webhook, y sea M el subconjunto de transacciones correspondientes al análisis de imágenes frontales (N_{front}) o traseras (N_{back}):

$$M = \{x \in \text{logs} \mid \text{node}(x) \in \{\text{'process_front'}, \text{'process_back'}\}\} \quad (2)$$

Definimos los indicadores de volumen del siguiente modo:

$$\text{Total Validations} = N_{\text{total}} \quad (3)$$

$$\text{Predicted Real} = \sum_{i \in M} (1 - I_{\text{fraude}}(S_i)) \quad (4)$$

$$\text{Predicted Fraud} = \sum_{i \in M} I_{\text{fraude}}(S_i) \quad (5)$$

Las tasas porcentuales de clasificación se formulan como:

$$\text{Percentage of Real (\%)} = \left(\frac{\text{Predicted Real}}{|M|} \right) \times 100 \quad (6)$$

$$\text{Percentage of Fraud (\%)} = \left(\frac{\text{Predicted Fraud}}{|M|} \right) \times 100 \quad (7)$$

4.3. Histogramas de Distribución de Scores

Para analizar la distribución probabilística del clasificador, se agrupan los scores continuos de cada transacción en $K = 10$ bins de ancho constante $W = 0,1$. El conteo de registros C_k en el bin k (donde $k \in \{1, 2, \dots, 10\}$) se define como:

$$C_k = |\{i \in M \mid S_i \in [(k-1) \cdot 0,1, k \cdot 0,1)\}| \quad (8)$$

(Para el último bin $k = 10$, el intervalo es cerrado por la derecha $[0,9, 1,0]$).

4.4. Promedios de Latencia y Tiempos de Respuesta

Sea $L_{AWS,i}$ la latencia en milisegundos de la transacción i en AWS Lambda, y sea $L_{Gemini,i}$ la latencia del servicio de humanización de Gemini. Los promedios aritméticos generales mostrados en el dashboard se calculan como:

$$\bar{L}_{AWS} = \frac{1}{|M|} \sum_{i \in M} L_{AWS,i} \quad (9)$$

$$\bar{L}_{Gemini} = \frac{1}{|M|} \sum_{i \in M} L_{Gemini,i} \quad (10)$$